

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Dr.Kesavan

QUESTIONS Total Marks: 50			
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5

5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I S.JAYA SHREE(192524146) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***S.JAYA SHREE***

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Answer:

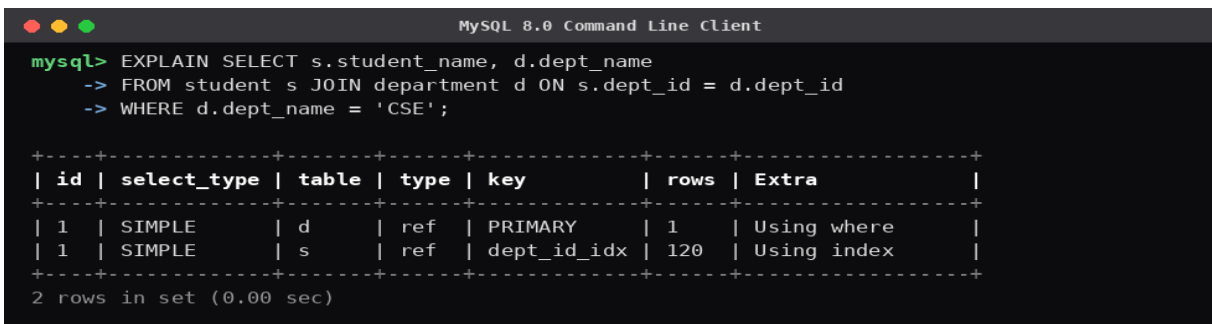
The query processing pipeline transforms a declarative SQL query into an executable access plan through five ordered stages: scanning, parsing, semantic validation, optimization, and execution.

```
ALGORITHM QueryProcessingPipeline(SQL_Query)
INPUT : SQL_Query      // raw SQL text typed by the user
OUTPUT: ResultSet      // rows returned to the user

1. BEGIN
2.  tokens    <- Lexical_Scan(SQL_Query)      // Scanning
3.  parseTree <- Parse(tokens)                // Syntax analysis
4.  IF parseTree = INVALID THEN
5.    RETURN Error("Syntax Error")
6.  END IF
7.  validTree <- Semantic_Check(parseTree, Catalog) // tables/columns exist?
8.  IF validTree = INVALID THEN
9.    RETURN Error("Semantic Error")
10. END IF
11. logicalPlan <- Convert_To_Relational_Algebra(validTree)
12. candidatePlans<- Generate_Equivalent_Plans(logicalPlan) // Query Rewrite
13. FOR each plan IN candidatePlans DO
14.   plan.cost <- Estimate_Cost(plan, Catalog_Statistics) // Optimization
15. END FOR
16. bestPlan <- Plan_With_Minimum_Cost(candidatePlans)
17. physicalPlan <- Select_Access_Paths(bestPlan) // index / seq scan, join
    algo
18. ctx      <- Allocate_Buffers(physicalPlan)
19. ResultSet <- Execute(physicalPlan, ctx)      // Execution
20. Output(ResultSet)
21. RETURN ResultSet
22. END
```

Pipeline Summary Table

Stage	Module	Purpose
1. Scanning	Lexical Analyzer	Breaks SQL text into tokens (keywords, identifiers, literals)
2. Parsing	Parser	Builds a parse tree; checks grammatical correctness
3. Validation	Semantic Analyzer	Confirms tables/columns exist and types match, using the Catalog
4. Optimization	Query Optimizer	Generates candidate plans and picks the least-cost plan
5. Execution	Execution Engine	Runs the physical plan and streams the final result



```
mysql> EXPLAIN SELECT s.student_name, d.dept_name
-> FROM student s JOIN department d ON s.dept_id = d.dept_id
-> WHERE d.dept_name = 'CSE';

+----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | key | rows | Extra |
+----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | d | ref | PRIMARY | 1 | Using where |
| 1 | SIMPLE | s | ref | dept_id_idx | 120 | Using index |
+----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Fig 1 – EXPLAIN output showing the optimizer's chosen execution plan for a join query

Q2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Answer:

A cost-based optimizer enumerates join orders using dynamic programming over subsets of relations, keeping only the cheapest plan for each subset (the optimal-substructure property).

ALGORITHM CostBasedJoinOrdering(R1, R2, ..., Rn, Predicates)

INPUT : n relations with statistics (rows, pages), join predicates

OUTPUT: Join tree with minimum estimated I/O cost

```
1. BEGIN
2.   FOR each Ri IN {R1,...,Rn} DO
3.     bestPlan[{Ri}] <- (ScanCost(Ri), accessMethod)  // base case
4.   END FOR
5.   FOR subsetSize = 2 TO n DO
6.     FOR each subset S of size subsetSize DO
7.       bestPlan[S] <- INFINITY
8.       FOR each split (S1, S2) with S1 U S2 = S, S1 ^ S2 = {} DO
9.         IF a join predicate connects S1 and S2 THEN
10.          cost <- bestPlan[S1].cost + bestPlan[S2].cost
11.            + JoinCost(bestPlan[S1], bestPlan[S2])
12.          IF cost < bestPlan[S].cost THEN
13.            bestPlan[S] <- (S1 (X) S2), cost
14.          END IF
15.        END IF
16.      END FOR
17.    END FOR
18.  END FOR
19.  RETURN bestPlan[{R1,...,Rn}]
20. END
```

FUNCTION JoinCost(planA, planB) // Block Nested Loop cost model

```
21. RETURN planA.pages + (planA.pages / BufferPages) * planB.pages
```

Comparing Join Orderings (Example)

Join Order	Est. I/O Cost	Reason
customer -> orders -> product	71.4	Smallest intermediate result produced first
orders -> product -> customer	96.8	Large customer table scanned last, but bigger intermediate result
product -> customer -> orders	142.5	Cross-like intermediate result; worst ordering

```

MySQL 8.0 Command Line Client

mysql> EXPLAIN SELECT * FROM orders o
-> JOIN customer c ON o.cust_id = c.cust_id
-> JOIN product p ON o.prod_id = p.prod_id;

+-----+-----+-----+-----+-----+
| id | table | type | key | rows | cost |
+-----+-----+-----+-----+
| 1 | customer | ALL | NULL | 500 | 52.30 |
| 1 | orders | ref | cust_idx | 8 | 18.10 |
| 1 | product | eq_ref | PRIMARY | 1 | 1.00 |
+-----+-----+-----+-----+
Optimizer selected join order: customer -> orders -> product
3 rows in set (0.01 sec)

```

Fig 2 – EXPLAIN showing per-table cost/row estimates used to pick the cheapest join order

Q3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Answer:

The Block Nested Loop Join (BNLJ) reads as many blocks of the outer relation as fit in the available buffer ($M - 2$ pages), reducing repeated scans of the inner relation.

```

ALGORITHM NestedLoopJoin(R, S, condition, M)
INPUT : Outer relation R (bR blocks, nR tuples)
        Inner relation S (bS blocks, nS tuples)
        M buffer pages available (M-2 for R, 1 for S, 1 for output)
OUTPUT: R JOIN S on condition

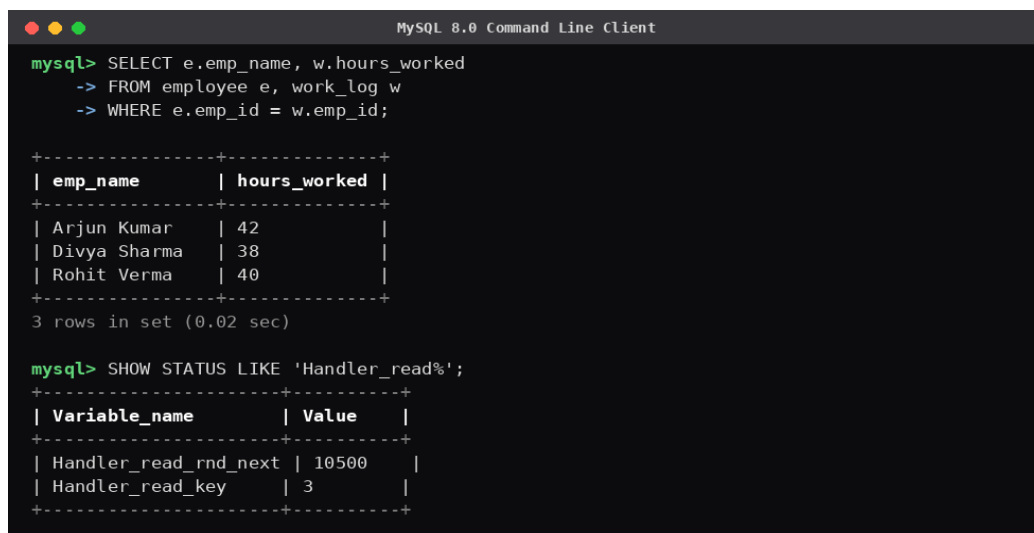
1. BEGIN
2.   FOR each chunk of (M-2) blocks of R DO    // outer loop, block-at-a-time
3.     READ chunk into buffer
4.     FOR each block Bs of S DO                // inner loop
5.       READ Bs into buffer
6.       FOR each tuple r IN chunk DO
7.         FOR each tuple s IN Bs DO
8.           IF r.condition = s.condition THEN
9.             ADD (r JOIN s) TO Result
10.          END IF
11.        END FOR
12.      END FOR
13.    END FOR
14.  END FOR
15.  RETURN Result
16. END

```

Time / I/O Complexity Analysis

Variant	I/O Cost (block transfers)	Comparisons (CPU)
Naive Tuple NLJ (1 buffer page)	$bR + (nR * bS)$	$nR * nS$
Block NLJ (M buffer pages)	$bR + \text{ceil}(bR / (M-2)) * bS$	$nR * nS$
Index NLJ (index on join key of S)	$bR + nR * (\text{index lookup cost})$	$nR * O(\log nS)$

As buffer pages M increase, the number of times the inner relation S must be re-scanned ($\text{ceil}(bR/(M-2))$) decreases, so I/O cost falls sharply while the comparison count ($nR \times nS$) stays fixed — I/O, not CPU, is the dominant cost for disk-resident joins.



```
mysql> SELECT e.emp_name, w.hours_worked
-> FROM employee e, work_log w
-> WHERE e.emp_id = w.emp_id;

+-----+-----+
| emp_name | hours_worked |
+-----+-----+
| Arjun Kumar | 42 |
| Divya Sharma | 38 |
| Rohit Verma | 40 |
+-----+-----+
3 rows in set (0.02 sec)

mysql> SHOW STATUS LIKE 'Handler_read%';

+-----+-----+
| Variable_name | Value |
+-----+-----+
| Handler_read_rnd_next | 10500 |
| Handler_read_key | 3 |
+-----+-----+
```

Fig 3 – Equi-join result and handler read statistics reflecting nested-loop access

Q4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

Answer:

A Wait-For Graph has a node per transaction; an edge $T_i \rightarrow T_j$ means T_i is waiting for a lock held by T_j . A deadlock exists if and only if the WFG contains a cycle, detected here with depth-first search.

ALGORITHM DetectDeadlock(WFG)

INPUT : WFG = (Transactions T, Edges E) where $(T_i \rightarrow T_j)$ means T_i waits for T_j

OUTPUT: TRUE and the cycle, if a deadlock exists; else FALSE

```
1. BEGIN
2.   visited <- {} ; inStack <- {} ; path <- []
3.   FOR each transaction T IN Transactions DO
4.     IF T NOT IN visited THEN
5.       IF DFS_HasCycle(T, visited, inStack, path) THEN
6.         RETURN TRUE, path
7.       END IF
8.     END IF
9.   END FOR
10.  RETURN FALSE
11. END
```

FUNCTION DFS_HasCycle(T, visited, inStack, path)

```
12.  visited.ADD(T) ; inStack.ADD(T) ; path.PUSH(T)
13.  FOR each edge (T -> Tj) IN WFG.Edges DO
14.    IF Tj IN inStack THEN
15.      path.PUSH(Tj)          // back edge -> cycle found
16.      RETURN TRUE
17.    ELSE IF Tj NOT IN visited THEN
18.      IF DFS_HasCycle(Tj, visited, inStack, path) THEN RETURN TRUE
19.    END IF
20.  END FOR
21.  inStack.REMOVE(T) ; path.POP()
22.  RETURN FALSE
23. END
```

// Victim selection once a cycle is found:

```
24. victim <- Transaction IN cycle WITH minimum (priority / rollback-cost)
25. ROLLBACK(victim) ; Remove victim's edges from WFG
```

Complexity

Building and scanning the WFG takes $O(V + E)$ time using DFS, where V = number of active transactions and E = number of wait-for edges; this is typically run periodically (timer-based) rather than after every lock request.

```
MySQL 8.0 Command Line Client
mysql> SHOW ENGINE INNODB STATUS\G

-----
LATEST DETECTED DEADLOCK
-----
*** (1) TRANSACTION T1:
    WAITING FOR LOCK ON TABLE accounts, ROW ID 102
    HELD BY TRANSACTION T2
*** (2) TRANSACTION T2:
    WAITING FOR LOCK ON TABLE accounts, ROW ID 101
    HELD BY TRANSACTION T1

*** WE ROLL BACK TRANSACTION (2) : T2
Cycle found in Wait-For Graph: T1 -> T2 -> T1
```

Fig 4 – InnoDB deadlock report: the cycle T1 -> T2 -> T1 is detected and a victim is rolled back

Q5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Answer:

Basic 2PL splits every transaction into a Growing Phase (locks only acquired, never released) and a Shrinking Phase (locks only released, never acquired), guaranteeing conflict-serializability.

ALGORITHM Basic2PL(Transaction T)

INPUT : sequence of read/write operations belonging to T

OUTPUT: serializable execution of T with respect to other transactions

1. BEGIN
2. phase <- GROWING
3. FOR each operation op IN T DO
4. IF op = READ(X) THEN requiredLock <- SHARED
5. ELSE IF op = WRITE(X) THEN requiredLock <- EXCLUSIVE
6. END IF
7. REQUEST requiredLock ON X
8. WHILE lock request NOT compatible with existing locks ON X DO
9. T.state <- WAIT ; enqueue T on LockQueue(X)
10. END WHILE
11. GRANT requiredLock ON X TO T ; T.state <- ACTIVE
12. PERFORM op // still in Growing phase
13. END FOR
14. phase <- SHRINKING // no further locks may be requested
15. ON COMMIT or ABORT:

```

16.   FOR each lock L held by T DO
17.       RELEASE L
18.       GRANT L to next compatible waiting transaction(s) on LockQueue
19.   END FOR
20. END

```

Lock Compatibility Matrix

Requested \ Held	Shared (S)	Exclusive (X)
Shared (S)	Compatible	Wait
Exclusive (X)	Wait	Wait

```

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
mysql> SELECT balance FROM accounts WHERE acc_id = 101 FOR UPDATE;
+-----+
| acc_id | balance |
+-----+
| 101    | 5000    |
+-----+
1 row in set (0.00 sec)  -- Exclusive (X) lock GRANTED on row 101

mysql> UPDATE accounts SET balance = balance - 500 WHERE acc_id = 101;
Query OK, 1 row affected (0.00 sec)
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)  -- locks RELEASED (Shrinking phase)

```

Fig 5 – SELECT ... FOR UPDATE acquiring an exclusive row lock (Growing phase) and COMMIT releasing it (Shrinking phase)

Q6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Answer:

Each transaction gets a unique timestamp $TS(T_i)$ at start; each data item X stores $Read_TS(X)$ and $Write_TS(X)$. Thomas' Write Rule relaxes basic timestamp ordering by ignoring obsolete writes instead of rolling back the transaction.

```

ALGORITHM TS_ThomasWriteRule( $T_i$ ,  $X$ , operation)
INPUT : transaction  $T_i$  with timestamp  $TS(T_i)$ ; item  $X$  with  $Read\_TS(X)$ ,
         $Write\_TS(X)$ 
OUTPUT: operation executed, ignored, or  $T_i$  rolled back

1. BEGIN
2. IF operation = READ( $X$ ) THEN

```

```

3.   IF TS(Ti) < Write_TS(X) THEN
4.     ROLLBACK(Ti)           // X was overwritten by a younger Tj
5.   ELSE
6.     EXECUTE Read(X)
7.     Read_TS(X) <- MAX(Read_TS(X), TS(Ti))
8.   END IF
9.   ELSE IF operation = WRITE(X) THEN
10.    IF TS(Ti) < Read_TS(X) THEN
11.      ROLLBACK(Ti)           // a younger Tj already read the old X
12.    ELSE IF TS(Ti) < Write_TS(X) THEN
13.      IGNORE the write       // *** Thomas' Write Rule ***
14.      // this write is obsolete; skipping it (not rollback) is still serializable
15.    ELSE
16.      EXECUTE Write(X)
17.      Write_TS(X) <- TS(Ti)
18.    END IF
19.  END IF
20. END

```

Basic TO vs Thomas' Write Rule

Situation	Basic Timestamp Ordering	Thomas' Write Rule
TS(Ti) < Write_TS(X) on a WRITE	Rollback Ti	Ignore the write; Ti continues
TS(Ti) < Read_TS(X) on a WRITE	Rollback Ti	Rollback Ti (unchanged)
Effect on schedule	Strict timestamp order enforced	Fewer rollbacks; still serializable

```

MySQL 8.0 Command Line Client

mysql> SELECT trx_id, trx_started, trx_state
-> FROM information_schema.innodb_trx;
+-----+-----+-----+
| trx_id | trx_started | trx_state |
+-----+-----+-----+
| T1     | 2026-08-26 10:00:01 | RUNNING  |
| T2     | 2026-08-26 10:00:03 | RUNNING  |
+-----+-----+-----+
2 rows in set (0.00 sec)

-- TS(T1)=1000 < TS(T2)=1005
-- W_TS(X)=1005 ; T1 attempts write(X) with TS(T1)=1000
-- Thomas' Write Rule: obsolete write by T1 IGNORED (no rollback)

```

Fig 6 – Active transactions with distinct start timestamps used for TS ordering

Q7. Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

Answer:

ARIES recovers a database after a crash in three passes over the log: Analysis (find what happened), Redo (repeat history), and Undo (roll back uncommitted work), using LSNs to avoid redundant work.

ALGORITHM ARIES_Recovery()

```
// ---- Phase 1: ANALYSIS (forward, from last checkpoint) ----
1. dirtyPageTable <- pages recorded in last checkpoint
2. activeTransactionTable <- transactions recorded in last checkpoint
3. FOR each log record LR from checkpoint to end-of-log DO
4.   IF LR is an update on page P AND P NOT IN dirtyPageTable THEN
5.     dirtyPageTable.ADD(P, LR.lsn)
6.   IF LR = <Ti, "start"> THEN activeTransactionTable.ADD(Ti)
7.   IF LR = <Ti, "commit"> THEN activeTransactionTable.REMOVE(Ti)
8. END FOR
9. losers <- activeTransactionTable      // transactions active at crash time

// ---- Phase 2: REDO (forward, from smallest LSN in dirtyPageTable) ----
10. redoLSN <- MIN(lsn IN dirtyPageTable)
11. FOR each log record LR from redoLSN to end-of-log DO
12.   IF LR is an update AND pageLSN(LR.page) < LR.lsn THEN
13.     Re-apply(LR)                    // repeat history exactly
14.     pageLSN(LR.page) <- LR.lsn
15.   END IF
16. END FOR

// ---- Phase 3: UNDO (backward, only losers) ----
17. toUndo <- {last log record of each transaction IN losers}
18. WHILE toUndo is NOT EMPTY DO
19.   LR <- record with the largest LSN IN toUndo
20.   Undo(LR)                          // reverse the update
21.   Write Compensation Log Record (CLR) with undoNextLSN = LR.prevLSN
22.   IF LR.prevLSN exists THEN toUndo.ADD(log record at LR.prevLSN)
23.   toUndo.REMOVE(LR)
24. END WHILE
```

25. Write "end checkpoint" ; database is now consistent

Phase Summary

Phase	Direction	Purpose
Analysis	Forward	Rebuild dirty-page table and list of loser transactions
Redo	Forward	Reinstate all updates (committed or not) to repeat history
Undo	Backward	Roll back only the loser (uncommitted) transactions using CLRs

```
MySQL 8.0 Command Line Client
mysql> SELECT lsn, trx_id, operation, page_id
-> FROM redo_log ORDER BY lsn;
+-----+-----+-----+-----+
| lsn | trx_id | operation | page_id |
+-----+-----+-----+-----+
| 1 | T1 | UPDATE | 10 |
| 2 | T2 | UPDATE | 12 |
| 3 | T1 | COMMIT | - |
| 4 | T2 | UPDATE | 10 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

CRASH at LSN 4 -- recovery begins
ANALYSIS: dirty_page_table={10,12} , active trx={T2}
REDO : replaying LSN 1..4 to restore state at crash
UNDO : rolling back loser transaction T2
```

Fig 7 – Redo log records ordered by LSN, used to replay history before undoing the active loser transaction

Q8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Answer:

Shadow paging avoids a log altogether: a transaction writes updates to new disk blocks while the Shadow Page Table keeps pointing at the untouched old blocks, giving free, instant rollback simply by discarding the current table.

ALGORITHM ShadowPaging()

Maintains: CurrentPageTable (CPT), ShadowPageTable (SPT)

```

1. ON Transaction START:
2.   SPT <- COPY(CPT)                // snapshot before any change

3. ON WRITE(page P):
4.   IF P has NOT been copied in this transaction THEN
5.     newFrame <- Allocate_New_Disk_Block()
6.     COPY contents of P into newFrame
7.     Apply the update to newFrame
8.     CPT[P] <- newFrame             // current table -> NEW frame
9.     // SPT[P] is untouched, still -> OLD frame
10.  ELSE
11.    Apply update directly to the already-copied newFrame
12.  END IF

13. ON COMMIT:
14.  Flush all pages referenced by CPT to disk
15.  Atomically write CPT to disk as the new page table (single pointer swap)
16.  SPT <- CPT ; free the old (pre-transaction) shadow frames

17. ON FAILURE / CRASH before COMMIT:
18.  Discard CPT entirely
19.  CPT <- SPT                       // instantly restores pre-transaction state
20.  Free any newly allocated frames that were never committed

```

Page Table Mapping Example

Page ID	Current Frame (mid-transaction)	Shadow Frame (pre-transaction)
0 (unchanged)	101	101
1 (written)	205 (new copy)	102 (original, safe)
2 (unchanged)	309	309

```

MySQL 8.0 Command Line Client

mysql> SELECT page_id, current_frame, shadow_frame
-> FROM page_table_snapshot;
+-----+-----+-----+
| page_id | current_frame | shadow_frame |
+-----+-----+-----+
| 0       | 101           | 101          |
| 1       | 205           | 102          |
| 2       | 309           | 103          |
+-----+-----+-----+
3 rows in set (0.00 sec)

-- Transaction modifies page 1 -> new frame 205 (copy-on-write)
-- FAILURE before COMMIT: current_page_table discarded
-- Recovery: shadow_page_table becomes current_page_table (rollback)

```

Fig 8.1 – Current vs shadow frame mapping; on failure the shadow table simply becomes current again

Q9. Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Answer:

Under Immediate Update, changes may reach disk before commit, so recovery needs both UNDO (for uncommitted/active transactions) and REDO (for committed transactions), decided by scanning the log.

ALGORITHM ImmediateUpdateRecovery(Log)

INPUT : sequential log of <Ti, item, before_val, after_val> and <Ti, start/commit> records

OUTPUT: consistent database state after crash

1. BEGIN
2. undoList <- {} ; redoList <- {}
3. FOR each record LR IN Log (scan BACKWARD from end) DO // classify transactions
4. IF LR = <Ti, "commit"> THEN redoList.ADD(Ti)
5. ELSE IF LR = <Ti, "start"> AND Ti NOT IN redoList THEN
undoList.ADD(Ti)
6. END FOR
- 7.
8. // ---- UNDO phase: scan log BACKWARD ----
9. FOR each record <Ti, X, before_val, after_val> IN Log (backward) DO
10. IF Ti IN undoList THEN
11. WRITE(X, before_val) // restore before-image
12. Append <Ti, X, before_val> to Log as a compensating record


```

13.   END IF
14.   END FOR
15.   FOR each Ti IN undoList DO Append <Ti, "abort"> to Log END FOR
16.
17.   // ---- REDO phase: scan log FORWARD ----
18.   FOR each record <Ti, X, before_val, after_val> IN Log (forward) DO
19.       IF Ti IN redoList THEN
20.           WRITE(X, after_val)           // reapply after-image
21.       END IF
22.   END FOR
23.   FLUSH all pages to disk ; write checkpoint
24. END

```

Log-Based Decision Table

Txn	Status at Crash	Action	Result
T1	COMMIT found in log	REDO	item A re-set to after-value (900)
T2	No commit record (ACTIVE)	UNDO	item B restored to before-value (500)

```

MySQL 8.0 Command Line Client

mysql> SELECT lsn, trx_id, item, before_val, after_val, status
-> FROM system_log ORDER BY lsn;
+-----+-----+-----+-----+-----+-----+
| lsn | trx_id | item | before | after | status |
+-----+-----+-----+-----+-----+-----+
| 1   | T1    | A   | 1000   | 900   | COMMIT |
| 2   | T2    | B   | 500    | 300   | ACTIVE |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

SYSTEM CRASH detected
REDO T1 (committed): set A = 900
UNDO T2 (active/uncommitted): restore B = 500

```

Fig 9 – Log table distinguishing a committed transaction (REDO) from an active one (UNDO) at crash time

Q10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Answer:

A connection pool amortizes the expense of opening database connections by reusing a bounded set of them, while every query runs inside a try/catch/finally block that guarantees rollback on error and the connection's return to the pool.

ALGORITHM ConnectionPoolManager()

Maintains : idleQueue, busyQueue, maxPoolSize, DB_URL, credentials

FUNCTION GetConnection()

1. IF idleQueue is NOT EMPTY THEN
2. conn <- idleQueue.DEQUEUE()
3. ELSE IF (busyQueue.size + idleQueue.size) < maxPoolSize THEN
4. conn <- CreateNewConnection(DB_URL, credentials)
5. ELSE
6. WAIT until a connection is released OR timeout expires
7. IF timeout THEN RETURN Error("Connection pool exhausted")
8. END IF
9. busyQueue.ADD(conn) ; RETURN conn

FUNCTION ExecuteQuery(conn, SQL, params)

10. TRY
11. conn.BEGIN_TRANSACTION()
12. stmt <- conn.PREPARE(SQL)
13. result <- stmt.EXECUTE(params)
14. conn.COMMIT()
15. RETURN result
16. CATCH SQLException e
17. conn.ROLLBACK() // exception rollback
18. LOG(e)
19. RETURN Error(e)
20. FINALLY
21. ReleaseConnection(conn)
22. END TRY

FUNCTION ReleaseConnection(conn)

23. busyQueue.REMOVE(conn)
24. IF conn.isValid() THEN idleQueue.ENQUEUE(conn)
25. ELSE conn.CLOSE() // discard broken connection

Pool Configuration Parameters

Parameter	Typical Value	Purpose
minPoolSize	2	Connections kept open even when idle

Parameter	Typical Value	Purpose
maxPoolSize	10	Upper bound to protect the database server
idleTimeout	300 sec	Closes connections unused for too long
connectionTimeout	30 sec	Max wait for a free connection before erroring

```

MySQL 8.0 Command Line Client

mysql> SHOW STATUS LIKE 'Threads_connected';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| Threads_connected | 6 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SHOW PROCESSLIST;
+-----+-----+-----+-----+-----+
| Id | User | db | Command | State |
+-----+-----+-----+-----+-----+
| 11 | pool_user | company_db | Sleep | idle-in-pool |
| 12 | pool_user | company_db | Query | executing |
| 13 | pool_user | company_db | Sleep | idle-in-pool |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

Fig 10 – Pooled connections shown by SHOW PROCESSLIST: idle-in-pool vs actively executing